ENVİRONMENT

ENVİRONMENT VARİABLES

ASPNETCORE_ENVIRONMENT DEĞİŞKENİ

RUNTİME ENVİRONMENT ORTAMINA ERİŞİM (IWEBHOSTENVİRONMENT)

ENVİRONMENT VARİABLE'LARA ERİŞİM

ENVİRONMNET VARİABLES'LARIN SECRETS.JSON VE APPSETTİNGS.JSON DOSYALARINI EZMESİ

Environment

Bir web uygulamasında, uygulamarın bulunduğu aşamalara göre çeşitli işlemler yapan bir yapıdır.

Projelerin geliştirme aşamalarında çeşitli durumlar bulunur. Bu durumlara göre farklı ortamlar vardır.

- Development
- Staging
- Productions

Development Environment: Projenin geliştirilirken kullanılır.

Staging Environment: Proje bittikten sonra çeşitli testler/hazırlık süreçleri için kullanılır.

Production Environment: Projenin yayınlandığı halde kullanılır.

Mesela Devepolment sürecinde, Production'da kullanılan Sql Server kullanılırsa işler bir birine girer. Bu yüzden Development ortamında projeye başka bir Sql server verilir, Stating için başka, Production için başka Sql server bağlantısı verilir.

Environment Variables

Projenin Runtime'da bulunduğu ortama göre davranışı belirleyen değişkenlerdir.

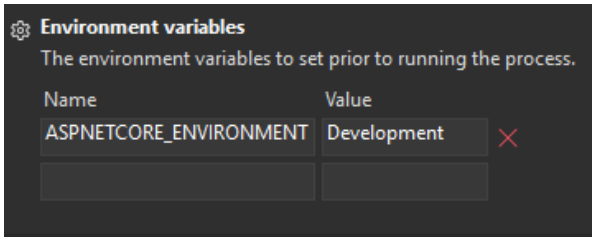
Projeye sağ tıklayıp properties seçeneği altındaki debug semesinden Environment Variables'a erişilebilir ve yeni değişkenler oluşturulabilir. Eskiden Direkt olarak debug sekmesi altında çıkıyordu artık debug semkesindeki link üzerinden Launch Profiles menüsüne yönlendiriliyor. Bu menü üzerinden erişiliyor.

Bu menü dışında Properties klasörü altındaki launchSettings.json dosyasından da Environmnet Variables'a erişilebilir.

ASPNETCORE_ENVIRONMENT Değişkeni

Mimari için bu değişken özel bir değişkendir. Bu Environment Variable, projenin hangi ortamda ayağa kalktığını mimariye bildirir.

Proje ilk oluşturulduğunda default olarak Development gelir.



Proje Publish edildiği zaman bu değişken otomatik olarak Production olur.

Staging yapmak için dümdüz Staging yazılır.

Runtime Environment Ortamına Erişim (IWebHostEnvironment)

Runtime'da projenin hangi ortamda çalıştığını anlamak için IWebHostEnvironment Interface'i kullanılır.

Bu Interface Asp.Net Core'da dahili olarak geldiği için istenilen yerde Dependecy Injection ile kullanılabilir.

DI ile gelen nesne üzerinden aktif olarak projenin çalıştığı ortam hakkında bilgi alınabilir. Mesela IsDevelopment fonksiyonu, proje Development ortamında ise True dönecektir.

```
_webHostEnvironment.IsDevelopment();
```

Aynı şekilde Production ve Staging için de benzer fonksiyonlar mevcuttur.

Bunun dışında Custom olarak oluşturulan, evrensel olmayan ortamlardan birinde ise proje IsEnvironmnet fonksiyonu ile öğrenilebilir.

```
_webHostEnvironment.IsEnvironment("Ahmet");
```

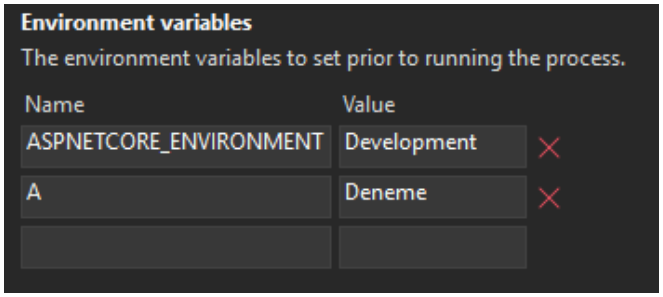
Proje Ahmet ortamında çalışıyorsa True dönecektir.

Environment Variable’lara Erişim

Environment Variables, secrets.json ve appsettings.json dosyasındaki değerler birbirlerinin yarine okunabilir ve aynı şekilde okunurlar.

Enviroment Variables’lar da IConfiguration ile erişilirler.

Örneğin A isiminde bir Environment Variable var.



Bu A değerini Controller’da çekmek için DI ile IConfiguration nesnesi alınır ve bu nesne üzerinden aynı appsettings.json veya secrets.json dosyasında oluduğu gibi istenilen değer çekilir.

```
var a =  _configuration["a"];
```

Environmnet Variables’ların secrets.json ve appsettings.json Dosyalarını Ezmesi

Bir Environment değişken, hem appsetting.json dosyasında hem de secrets.json dosyasında varsa ne olur ?

IConfiguration üzerinden bir veri isteği yapıldığında(örn `_configuration["a"];`) mimari önce Environment Variables’a bakar, eğer böyle bir değer yoksa secret.json’a bakar eğer burda da ilgili değer yoksa appsettings.json dosyasına bakar.

Yani ilgili sırlama ile dosyalar birbirlerini ezeler. Birinde bulunması halinde aranan değer diğerine hiç bakılmaz.

IConfiguration configuration
configuration["a"]
Environment Vari.
Secrets.Json
appSettengs.Json